경력기술서

임기호

백엔드 개발자

프로젝트 포트폴리오 ↗

lasid84@gmail.com

010-5496-0663

GitHub ↗

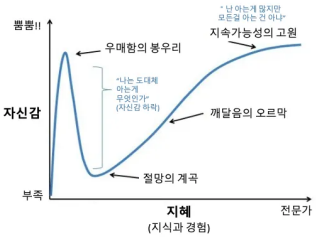
00 소개

업무의 의미가 드러나고, 안심하고 바꿀 수 있는 코드를 지향합니다.

비슷해 보이는 코드라도 업무에서 맡는 책임과 변경 이유를 살펴 공통화합니다. 함께 바뀌어야 하는 것은 모으고, 업무마다 달라지는 판단은 분리합니다.

테스트로 기존 업무 동작이 유지되는지 확인하며 구조를 개선합니다. 운영 중 반복되는 문제와 변경하기 어려운 부분을 찾아, 팀이 이해하고 고쳐 나갈 수 있는 코드로 다듬습니다.

문제를 해결할 때는 제 판단도 검증의 대상으로 둡니다. 예상과 실제 동작이 다르면 로그와 실행 환경을 다시 확인하고, 근거에 따라 판단을 바꿉니다.



우매함의 봉우리를 경계하며, 모르는 것을 인정하고 계속 배우려 합니다.

01 경력

(주)케이더블유이코리아

국제 종합 물류 기업

디지털솔루션부 차장 (Lead Software Engineer)

2022.01 ~ 현재

www.kwe.com ↗

항공·해운·재무 업무를 다루는 사내 시스템의 설계·개발·운영을 리드합니다. 3인 개발팀에서 주간 코드 리뷰와 개발 기준 정립을 담당합니다.

- 외부 시스템 연동의 전체 구조를 설계하고 동료와 기능을 나눠 분석·구현했습니다.
- API/Worker 실행 구조와 CI/CD를 단독으로 구성했습니다.
- 현업과 데이터 기준을 정리하고 운영 장애의 원인을 분석해 개선했습니다.

ILJIN Global

베어링 제조 기업

MES 개발팀 과장 (Software Engineer)

2010.12 ~ 2022.01

www.iljin.com ↗

생산관리시스템(MES)을 개발하고 운영했습니다. 국내외 공장의 MES 리뉴얼, SAP 연동, 생산·납품·재고 데이터 처리와 설비 알림 시스템을 담당했습니다.

02 최근 주요 프로젝트

통관 서류 검증 자동화

Local LLM, OCR(Tesseract), Vitest

(주)케이더블유이코리아

2026.06 ~ 2026.09

담당

문서 검증·입력·회신 업무 자동화 개발

미국·중국 통관 서류 9종의 55개 항목을 담당자가 수작업으로 대조하고 있어, 서류 간 누락과 불일치를 자동으로 확인하는 검증 시스템을 개발했습니다.

업체마다 다른 양식에 대응하기 위해 문서별 추출 항목의 설명과 few-shot 예시를 설정으로 관리했습니다. 파일 형식별 데이터 추출과 LLM 호출을 분리하고, 추출한 텍스트를 문서별 설정과 함께 LLM에 전달해 필요한 항목을 구조화했습니다.

추출·검증 결과는 사용자가 확인하는 단계를 거치도록 했습니다. 처리 중 발생한 오류는 테이블에 기록하고, 원인을 분석해 관련 로직을 수정하고 있습니다.

- 통관 서류 **9종의 55개 대조 항목**에서 누락과 불일치를 자동 탐지했습니다.
- 현업 확인 기준 **통관 1건당 서류 검토 시간을 8분에서 3분**으로 줄였습니다.
- 시스템 등록·회신 메일 작성·파일 첨부도 자동화해 **후속 업무에서 하루 약 1시간을 추가 절감**했습니다.

웹 기반 통합 사내 시스템 구축

Express, Next.js, Turborepo, PostgreSQL, PL/pgSQL, PM2, Worker Threads

(주)케이더블유이코리아

2023.09 ~ 현재

담당

전체 설계·개발 리드, 현업과 기준정보 정제

입사 당시 핵심 담당자 두 명이 이탈한 상태에서, 여러 언어와 기술로 개발된 100개 이상의 프로그램 소스코드를 인계받았습니다. 시스템이 나뉘어 있어 개선 요청을 반영하기 어려웠고, 운영 문제는 데이터 패치로 대응하는 경우가 많았습니다. 특정 담당자가 없어도 팀이 시스템을 이해하고 개선을 이어갈 수 있도록 웹 플랫폼 통합을 추진했습니다.

- **15종 이상의 업무 시스템**을 하나의 웹 플랫폼으로 통합하고 있습니다. 한 부서에 적용한 본사 시스템 연동과 통관 정보 조회 기능을 다른 부서에서도 활용할 수 있게 됐습니다.
- 5개 업무부가 개별 관리하던 거래처 정보 약 3만 건을 현업과 검토·정제하고 글로벌 코드 기준으로 통합했습니다.
- 관세청 API와 스크래핑을 연동해 건당 약 10분 걸리던 조회를 **5초 이내**로 줄였습니다.
- C# 기반 입력 자동화를 웹 플랫폼으로 통합하고, 공용 라이브러리로 중복 코드를 정리했습니다.

그룹웨어 결재로 개선 요청을 접수하고 검토 후 개발에 반영하는 절차를 마련했습니다. 개별 데이터를 직접 수정하던 대응에서 프로그램을 개선하는 방식으로 운영을 바꿨습니다.

이후 NestJS로 백엔드를 전환하고 컨테이너 기반 배포 체계를 구축했습니다.

글로벌 업무 시스템 연동 자동화

(주)케이더블유이코리아 · 통합 플랫폼 개선 사례

통합 플랫폼 구축·운영 중 개선

담당

전체 구조 설계, 동료 1명과 외부 호출 75개 공동 분석·구현

Node.js, TypeScript, HTTP, Puppeteer, Redis, PostgreSQL

현업에서는 하루 약 **800~900건의 운송장** 정보를 내부 시스템과 해외 본사 업무 시스템에 반복 입력했습니다. 처음에는 본사 시스템 변경 시 화면을 기준으로 대응하기 쉽다고 판단해 Puppeteer로 자동화했지만, 화면 자원 로딩이 **100Mbps 사내 회선의 60~70Mbps를 점유**했습니다.

- 본사 웹 화면의 HTTP 요청·응답과 인증 과정을 분석하고, 브라우저가 보내던 요청을 Node.js · TypeScript 코드로 재현했습니다. 총 **75개 HTTP 요청을 구현해 12종 업무 흐름**을 전환했습니다.
- 운송장 데이터의 **400개 이상 필드**를 분석하고, 요청 구성과 응답 해석 로직을 여러 업무에서 재사용하도록 만들었습니다.
- 직접 구성한 요청의 처리 결과와 실제 화면 입력 후 계산된 결과를 각각 저장하고 비교해, 기존 업무와 같은 결과가 나오는지 확인했습니다.
- 두 백엔드 컨테이너가 **Redis에 저장한 세션을 공유**하도록 구성했습니다. 인증 락을 획득한 뒤 세션을 다시 확인해 다른 요청이 갱신한 세션을 재사용했습니다.
- 사용자 요청과 자동 배치에서 같은 인증 처리와 업무 API를 재사용하도록 구성했습니다.
- 전환 후 운영에서 관측한 네트워크 사용량은 **60~70Mbps에서 대체로 5Mbps 미만**으로 줄었습니다.

Node.js API 서버 메모리 장애 개선

(주)케이더블유이코리아 · 통합 플랫폼 개선 사례

2026.08

담당

메모리 장애 분석과 조회 구조 개선

OOM 분석 회고 ↗

Node.js, V8, Prisma, PostgreSQL, 메모리 계측

운영 중인 Node.js API 서버가 메모리 부족으로 반복 종료됐습니다. OCR 처리와 Prisma 쿼리 엔진을 의심했지만 관련 조치 후에도 장애가 이어져 로그와 실제 배포 버전, 조회 조건을 다시 확인했습니다.

- 기간 조건에 해당하는 데이터를 **수만 건씩 한 번에 조회하고 응답으로 가공**하는 경로를 찾았습니다.
- 요청당 처리량을 줄이기 위해 **500건 단위 조회와 무한스크롤**을 적용했습니다.
- 조회 구조를 개선한 뒤, 응답 처리 과정을 별도로 모사해 객체 변환·JSON 직렬화·Buffer 생성 단계의 **힙 사용량(heapUsed)과 RSS**를 측정했습니다. 반복 실행에서도 GC 이후 힙 사용량은 누적되지 않았고 RSS는 초기 증가 이후 일정 범위에 수렴했습니다. 모사한 응답 처리 경로에서는 지속적인 누수 징후가 관측되지 않았습니다.
- 조회 구조 개선 이후 동일 장애는 재발하지 않았습니다.

Docker Swarm 기반 CI/CD 표준 배포 체계 구축

(주)케이더블유이코리아

2025.11 ~ 2025.12

담당

CI/CD 설계·구축 및 후속 개선 단독 수행

GitLab CI/CD, Docker, Docker Swarm, Portainer

기존 PM2 중심의 수동 배포는 서버 상태와 환경 차이에 따라 결과 재현이 어려웠고, 테스트 누락 방지와 배포 이력 추적·롤백 기준도 미흡했습니다.

- 테스트 > 빌드 > 배포 파이프라인을 구성하고 테스트 실패 시 후속 단계를 차단했습니다.
- 브랜치·SHA 태그와 디제스트로 이미지를 식별해 동일 이미지를 다시 배포할 수 있도록 했습니다.
- 빌드·배포 로그와 아티팩트를 남겨 배포 이력과 실패 단계의 로그를 확인할 수 있도록 구성했습니다.
- Portainer를 도입해 서비스·태스크 상태와 컨테이너 로그를 한곳에서 확인하는 모니터링 환경을 구축했습니다.

2026년 9월에는 Kaniko에서 buildx로 전환하고 러너·캐시 구성을 개선했습니다. 앱별 잡을 한 파이프라인에 모으고 변경 경로에 따라 실행하도록 구성했습니다. 테스트·빌드·배포 전체 파이프라인 시간을 **약 12분에서 6분**으로 줄였습니다.

백엔드 서비스 리팩토링(Express > NestJS)

(주)케이더블유이코리아

2025.06 ~ 2025.10

담당

도메인 구조 정리·팀 개발 기준 정립,
API/Worker 구조 단독 설계

Node.js, NestJS, Prisma, PostgreSQL, Redis, BullMQ

기능이 늘면서 모듈 간 의존성이 복잡해지고 일관된 코드 구조 유지와 단위 테스트 적용이 어려워졌습니다.
NestJS로 전환하며 도메인별 모듈과 의존성을 정리하고, 팀 코드 컨벤션과 TDD 기반 개발 환경을 구축했습니다.

- 코드베이스를 도메인별 모듈로 나눠 신규 기능 개발과 유지보수에 필요한 구조를 정리했습니다.
- Redis와 BullMQ로 무거운 작업을 비동기 배치 파이프라인으로 분리하고, 인증 데이터를 캐싱해 DB 부하를 줄였습니다.
- DI와 데코레이터 등 NestJS 구조를 도입해 인증·로깅 등 공통 로직을 재사용하도록 했습니다.
- 단위·통합 테스트로 변경 영향 범위를 사전에 검증할 수 있는 구조를 마련했습니다.

업무 로직은 API에서 실행하고, BullMQ Worker는 작업 실행 요청과 재시도를 담당하도록 구성했습니다.

같은 업무가 동시에 중복 등록되지 않도록 업무 식별 키별 PostgreSQL advisory lock을 획득하고, 같은 트랜잭션에서 진행 중인 작업을 확인한 뒤 없을 때만 등록했습니다.

작업별 재시도 횟수와 간격을 정하고, 재시도 대기와 최종 실패를 별도 상태로 기록했습니다. 오류 원인과 요청 데이터를 남겨 지정한 작업을 다시 실행할 수 있도록 했습니다. 등록 요청 시 DB에는 진행 중이지만 큐에는 없는 작업을 확인하면 기존 작업을 실패 처리하고 재등록하도록 했습니다.

RPA 프로젝트 (삼성 NERP 연동)

(주)케이더블유이코리아

2024.08 ~ 2024.12

담당

문서 검증·데이터 입력 자동화 개발

C# (.NET 4.5), WinForms, OCR, MySQL

삼성 시스템의 데이터 입력 작업에 시간이 많이 소요되어 업무팀의 야근이 빈번하게 발생했습니다. 문서 검증과 입력 과정을 자동화했습니다.

- 문서 검증·입력 자동화로 **월 1,200시간을 절감**했습니다(기존 경력기술서 산정치).
- 실시간 처리로 입력 누락과 오입력 가능성을 줄였습니다.

세금계산서 프로그램 리뉴얼

(주)케이더블유이코리아

2023.01 ~ 2023.07

담당

업무 로직 재구현·DB 전환·데이터 이관 및 검증

C# (.NET 6), gRPC, WinForms, PostgreSQL, PL/pgSQL

Oracle Forms 버전의 기술지원 종료로 유지보수가 어려워졌고, Oracle 라이선스 비용과 확장 제약을 줄이기 위해 .NET 6 기반 리뉴얼과 PostgreSQL 전환을 추진했습니다.

- Oracle Forms 기반 세금계산서 업무 로직 전체를 PL/pgSQL로 재구현했습니다.
- Oracle에서 PostgreSQL로 전환해 라이선스 비용을 줄였습니다.
- **1,000만 건의 데이터 이관을 자동화**하고 정합성을 검증했습니다.
- 스마트빌 연동으로 국세청 즉시 신고 및 매입 데이터를 실시간으로 처리했습니다.

03 제조·MES 프로젝트

운반비 전산화 구축

ILJIN Global

2020.04 ~ 2020.07

담당

운반비 업무 전산화·SAP 연동 개발

C# (.NET 4.5), ASP.NET, WinForms, WCF, Oracle, PL/SQL

수기 작성·수작업 중심의 운반비 관리를 전산화했습니다.

- 8개 공장의 운반비 프로세스를 전산화해 수작업을 줄이고 처리 절차를 통일했습니다.
- SAP 연동으로 운반비를 실시간으로 처리하고 비용 투명성을 확보했습니다.
- 최우수사원 표창을 받았습니다.

MES – SAP 연동 인터페이스 개발

ILJIN Global

2019.01 ~ 2019.12

담당

MES·SAP 실적 및 기준정보 인터페이스 개발

C# (.NET 4.5), ASP.NET, WinForms, WCF, Oracle, PL/SQL

생산·납품·재고 정보를 수작업으로 전송하고 중복 입력하면서 생기는 지연과 오류 가능성을 줄이기 위해 실시간 연동 인터페이스를 개발했습니다.

- ERP(SAP)와 MES 사이의 생산·납품·재고 이동 데이터를 실시간으로 연동했습니다.
- 기준정보를 자동 이관해 운영 오류와 수작업을 줄였습니다.

제천·슬로바키아 공장 MES 업그레이드

ILJIN Global

2016.04 ~ 2017.12

담당

국내외 공장 MES 리뉴얼·LOT 추적 시스템 개발

VB6 > C# (.NET 4.5), ASP.NET, WinForms, WCF, Oracle 9i > 11g, PL/SQL

VB6 기반 MES의 기술지원 종료와 컴포넌트·개발 환경 제약을 해결하기 위해 C# 기반 리뉴얼과 데이터베이스 업그레이드를 수행했습니다.

- 제천·슬로바키아 공장의 MES를 리뉴얼했습니다.
- LOT 단위 추적 시스템을 구축해 품질 관리 프로세스를 개선했습니다.
- 과장으로 특진했습니다.

SPC 이상현상 알림 시스템 개발

VB6, Oracle PL/SQL, TCP Socket

ILJIN Global

2015.09

담당

TCP 소켓 기반 설비 알림 모듈 단독 설계·개발

생산 품질 이슈를 실시간으로 감지하고 알리는 SPC 시스템을 구축했습니다. 공정 설비와 연계하는 TCP 소켓 기반 알림 모듈을 단독으로 설계·개발했습니다.

- 일평균 100만 건의 생산 데이터 환경에서 SQL로 7가지 이상 징후 판정 로직을 구현했습니다.
- SMS와 현황판으로 알림을 전달했습니다.

04 기술

백엔드

Node.js, TypeScript, Express, NestJS, Prisma, Redis, BullMQ
C# (.NET Framework, .NET 6), gRPC, WCF

물류 플랫폼의 Express·NestJS 서버, Redis 기반 인증 조정과 BullMQ 배치 처리에 사용했습니다. C#은 제조 MES와 물류 업무 도구, gRPC·WCF는 시스템 간 연동에 사용했습니다.

프론트엔드

Next.js, React, React Native, Expo, WinForms

사내 웹 플랫폼과 개인 앱의 화면을 구현했습니다. WinForms로 제조·물류 업무 도구를 개발했습니다.

데이터

PostgreSQL, Oracle, MySQL, PL/SQL, PL/pgSQL

업무 로직과 인터페이스를 구현하고 Oracle > PostgreSQL 전환에서 1,000만 건을 이관·검증했습니다.

배포·운영

GitLab CI/CD, Docker, Docker Swarm, buildx, Portainer, PM2, Turborepo

초기 PM2 운영에서 컨테이너 배포로 전환하고 2026년 9월 빌드·러너·캐시를 개선했습니다.

문서 처리·검증

Local LLM, Tesseract OCR, Vitest

통관 문서의 누락·불일치 검증과 데이터 추출에 사용하고 단위·통합 테스트로 변경 영향을 확인했습니다.

05 개인 프로젝트

댓글 필터 (Android · Web)

React Native, Expo, TypeScript, WebView, Next.js, Transformers.js, WebLLM, Web Worker

Google Play ↗ 웹 버전 ↗ 개발 과정 ↗

앱 기획부터 구현과 배포까지 혼자 수행했습니다. 앱에서는 네이버 블로그 글을 WebView로 열어 온디바이스에서 댓글 가시성을 조절하고, 좋아요 기준 필터와 차단 사용자 숨김 기능을 적용했습니다.

Next.js 웹 버전에서는 Transformers.js와 WebLLM을 Web Worker로 구동해 브라우저 온디바이스에서 댓글을 분류하고 본문을 요약합니다.

HealthDog

React Native, Expo, TypeScript, Google OAuth, Firebase, Supabase, AdMob, RevenueCat

Google Play ↗

운동 기록·조회·통계 기능 중심의 앱을 혼자 개발했습니다. Google OAuth 로그인, Supabase·Firebase 데이터 저장 및 동기화, AdMob 광고와 RevenueCat 구독 결제 등 운영에 필요한 외부 서비스를 연동했습니다.

액션독: 3라인 퍼즐 (Android)

React Native, Expo, TypeScript, Google OAuth, Supabase, AdMob

Google Play ↗

3라인 매치 퍼즐 게임을 혼자 기획·개발하고 스토어에 출시했습니다. 블록 하강·매치 판정·콤보 연쇄·게임오버 등 게임 코어 엔진을 도메인 주도 아키텍처의 4계층으로 나눠 직접 설계·구현했습니다.

Google OAuth 로그인, Supabase 기반 글로벌 점수 랭킹, AdMob 광고를 연동했습니다.